

DOCUMENTO TÉCNICO EXECUTIVO

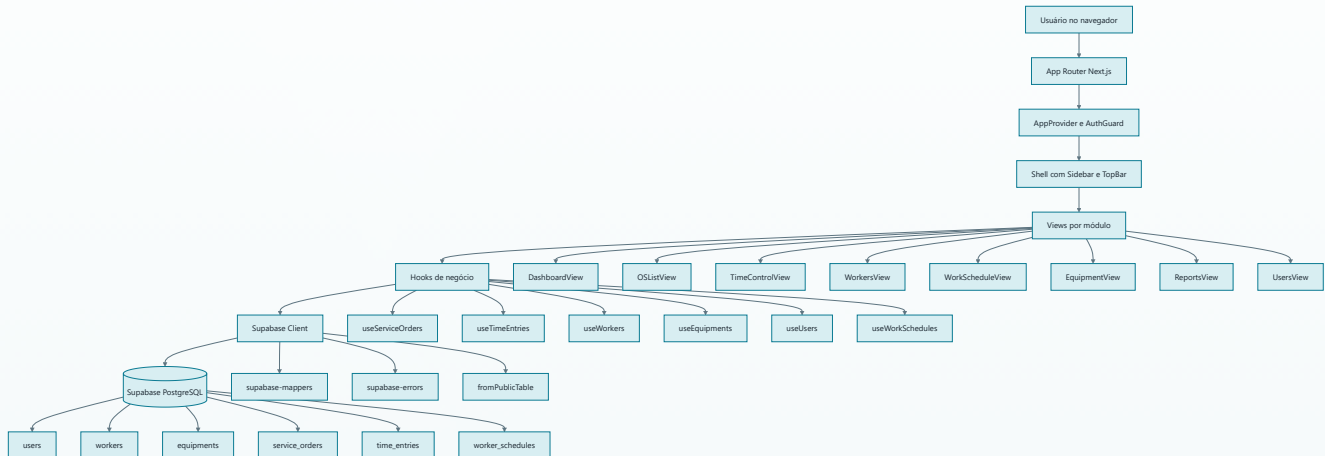
Fluxograma completo do projeto Manutenção ITA e seus relacionamentos com a base de dados

Este documento consolida a arquitetura do sistema, a navegação funcional, os fluxos críticos de negócio, a camada de autenticação e permissões, e o modelo relacional do banco Supabase/PostgreSQL utilizado pela aplicação.

1. Visão geral da solução

O sistema foi estruturado em quatro camadas principais: apresentação, lógica de aplicação, integração com dados e banco relacional. A navegação é protegida por autenticação local validada no Supabase, e cada módulo usa hooks especializados para realizar operações CRUD, compor dados e sincronizar a interface com eventos de atualização.

ARQUITETURA PONTA A PONTA



Resumo executivo

- A aplicação web usa o App Router do Next.js para organizar rotas protegidas e módulos funcionais.
- O contexto global controla login, hidratação de sessão e usuário corrente por meio de localStorage e validação na tabela users.
- As telas principais são componentes container que consomem hooks dedicados por domínio.
- Os hooks concentram leitura, escrita, tratamento de erro, mapeamento de tipos e atualização reativa.
- O Supabase funciona como gateway de acesso ao PostgreSQL e também fornece canais de realtime para recarga automática.

App Router

Context API

CRUD desacoplado

Realtime

Tipagem forte

2. Módulos funcionais da aplicação

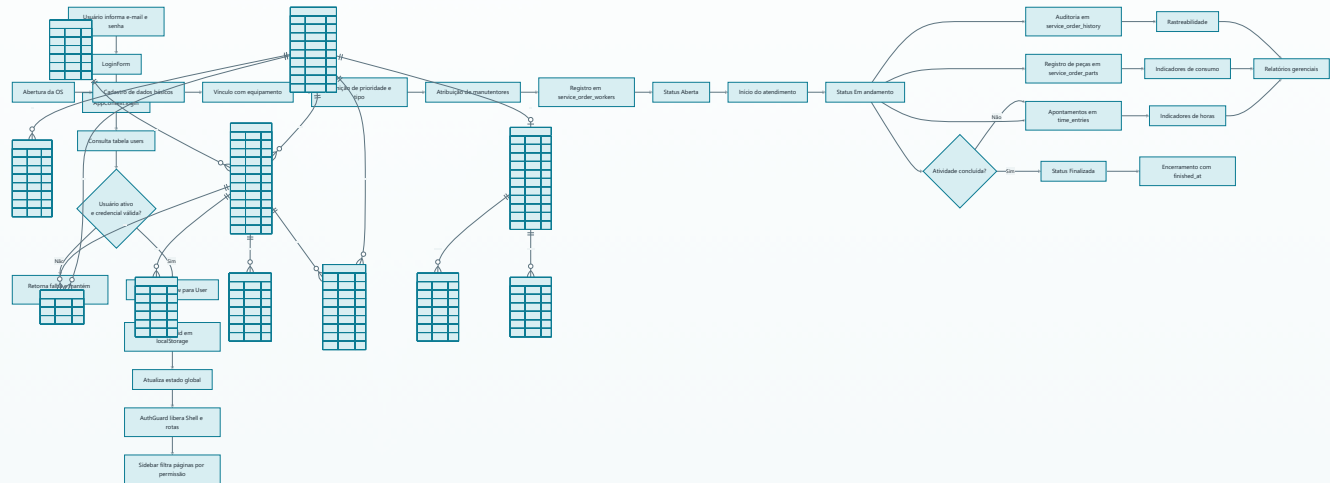
Cada rota principal do sistema representa um domínio operacional. A tabela abaixo conecta navegação, componente principal, hooks utilizados e tabelas impactadas.

ROTA	MÓDULO	COMPONENTE PRINCIPAL	HOOKS DOMINANTES	TABELAS MAIS USADAS
/	Dashboard	DashboardView	useServiceOrders, useWorkers, useEquipments, useTimeEntries	service_orders, workers, equipments, time_entries
/ordens	Ordens de Serviço	OSListView	useServiceOrders, useWorkers, useEquipments	service_orders, service_order_workers, service_order_parts, service_order_history
/tempo	Controle de Tempo	TimeControlView	useTimeEntries, useServiceOrders, useWorkers	time_entries, service_orders, workers
/manutentores	Equipe técnica	WorkersView	useWorkers	workers
/escalas	Escalas e disponibilidade	WorkScheduleView	useWorkSchedules, useWorkers	worker_schedules, worker_schedule_day_overrides, worker_schedule_cycle_overrides
/equipamentos	Ativos e linhas	EquipmentView	useEquipments	equipments, service_orders
/relatorios	Análises gerenciais	ReportsView	useServiceOrders, useWorkers, useEquipments, useTimeEntries	service_orders, service_order_parts, time_entries, workers, equipments
/usuarios	Administração de acesso	UsersView	useUsers, useWorkers	users, workers
/configuracoes	Parâmetros e preferências	SettingsView	Contexto e permissões	users e regras de acesso

3. Fluxo de autenticação, sessão e permissões

O acesso ao sistema é controlado pelo contexto global da aplicação. A sessão não é persistida pelo módulo de auth do Supabase; em vez disso, o sistema armazena o identificador do usuário em localStorage e revalida esse vínculo a cada carregamento da interface.

FLUXO DE LOGIN E GUARDA DE ROTA



Perfis suportados

- Administrador: acesso total a operações, usuários, relatórios e configurações.
- Supervisor: foco operacional, sem gestão completa de usuários.
- Manutentor: atua sobre OS próprias e apontamento de tempo.
- Operador: mesmo perfil funcional do manutentor, com vínculo à tabela workers.

Controle de acesso

- As rotas exibidas no menu são filtradas por hasPermission.
- As permissões são centralizadas em lib/permissions.ts.
- O vínculo user.worker_id limita a visualização de OS e apontamentos próprios.
- Usuários inativos são bloqueados tanto no login quanto na hidratação da sessão.

4. Fluxo operacional central do sistema

O coração do projeto é a gestão de ordens de serviço. Esse processo conecta equipamentos, equipe técnica, histórico, peças consumidas, apontamento de tempo e relatórios gerenciais.

CICLO DE VIDA DA ORDEM DE SERVIÇO

Como os dados são montados

O hook `useServiceOrders` busca, em paralelo, `service_orders`, `service_order_workers`, `service_order_parts`, `service_order_history` e `time_entries`. Em seguida, compõe um objeto de domínio enriquecido para cada OS.

Valor técnico dessa abordagem

A composição fora do banco simplifica a camada de visualização, preserva tipagem consistente e concentra a regra de negócio em um ponto único da aplicação.

5. Arquitetura de dados e relacionamento com o banco

A base Supabase organiza o domínio em torno da tabela `service_orders`, que funciona como entidade central do processo de manutenção. Usuários, mantenedores, equipamentos, apontamentos, peças e escalas se relacionam de forma explícita por chaves estrangeiras.

MODELO RELACIONAL PRINCIPAL

Os relacionamentos foram definidos com regras de integridade coerentes com o domínio: exclusão restrita para equipamentos com OS associadas, exclusão em cascata para vínculos e históricos subordinados, e nullificação controlada em associações opcionais como `users.worker_id` e `service_orders.assigned_to`.

6. Tabelas, responsabilidades e impacto funcional

A seguir está o papel técnico de cada tabela e o efeito direto que ela produz na aplicação.

Autenticação e acesso

- users: guarda credenciais, perfil de acesso, flag de ativo e vínculo opcional com workers.
- workers: representa o colaborador operacional, sua especialidade, turno e capacidade diária.

Cadastro de ativos

- equipments: cadastro de máquinas, localização e linha produtiva.
- service_orders: entidade central do processo de manutenção.

Execução operacional

- service_order_workers: resolve o relacionamento N para N entre OS e trabalhadores.
- time_entries: mede esforço, improdutividade, setup e deslocamento por OS e por manutentor.
- service_order_parts: controla consumo de peças e subsidia relatórios de custo.

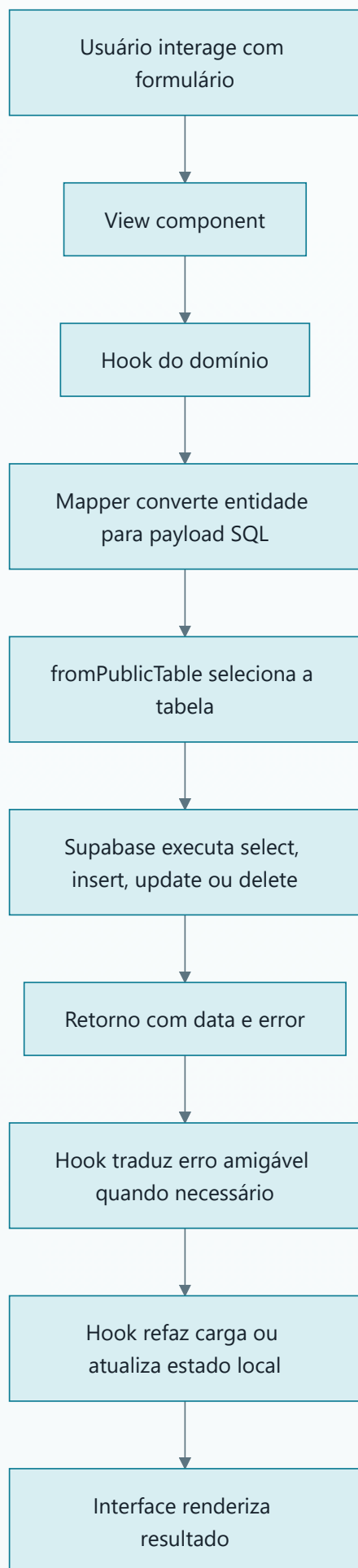
Rastreabilidade e escala

- service_order_history: registra mudanças relevantes de estado e atribuição.
- worker_schedules: define o ciclo-base da escala do colaborador.
- worker_schedule_day_overrides e worker_schedule_cycle_overrides: tratam exceções operacionais.

7. Fluxo técnico entre interface, hooks e Supabase

O padrão repetido em quase todo o sistema segue uma cadeia previsível: view, hook, mapper, cliente Supabase e tabela de destino. Isso reduz acoplamento entre interface e modelo físico do banco.

PADRÃO DE EXECUÇÃO DE UM CRUD



Aspectos arquiteturais relevantes

- Os mappers isolam as diferenças entre nomes de coluna do banco e o shape usado na UI.
- As views permanecem focadas em estado visual, filtros, formulários e ações do usuário.
- Os hooks concentram as operações assíncronas e a atualização da coleção em memória.
- Parte dos domínios usa canais postgres_changes do Supabase para sincronização quase em tempo real.
- O tratamento de erro usa mensagens amigáveis para violações de unicidade e integridade referencial.

8. Pontos fortes e observações estruturais

Além da visão de fluxo, vale destacar alguns comportamentos que influenciam a operação e a evolução futura do sistema.

Pontos fortes

- Boa separação entre tela, regra de negócio e persistência.
- Modelo relacional compatível com rastreabilidade operacional.
- Escalas e exceções suportam cenários reais de manutenção industrial.
- Permissões por perfil estão centralizadas e legíveis.
- Relatórios derivam diretamente do dado operacional, sem duplicação de fonte.

Observações técnicas

- O login atual consulta a tabela users diretamente e não usa autenticação gerenciada do Supabase Auth.
- useServiceOrders faz composição em memória a partir de múltiplas consultas paralelas.
- A tabela service_orders é o maior hub de dependência do banco.
- A exclusão de equipamentos é protegida tanto por regra de negócio quanto por integridade referencial.
- Os relatórios dependem fortemente da qualidade dos apontamentos em time_entries.

Documento gerado a partir da análise do código-fonte e do schema SQL do projeto Manutenção ITA em 09/04/2026.